arXiv:2611.08918v1 [cs.LG] 6 Nov 2026

Composable Causality: A Toolkit for Systematic Time-Series Causal Discovery and Treatment-Effect Benchmarking

Muhammad Hasan Ferdous
Causal AI Lab
Department of Information Systems
University of Maryland, Baltimore County
h.ferdous@umbc.edu

Md Osman Gani
Causal AI Lab
Department of Information Systems
University of Maryland, Baltimore County
mogani@umbc.edu

Abstract

Real-world multivariate time series rarely come with verified ground-truth causal structure, so research on time-series causal discovery and treatment-effect estimation depends on synthetic data. Existing tools are either fixed datasets that cannot be modified after release or script-based generators in which each script fixes one combination of mechanism, confounding, and missingness. Neither supports controlled experiments that vary a single property, such as sparsity, noise distribution, or missingness mechanism, while holding the others fixed. Counterfactual evaluation has the same gap, since existing simulators are built per paper with hardcoded mechanisms or produce observational data only. We introduce DTCD[1] (Datasets and Toolkit for Causal Discovery), a data generation library that produces multivariate time series with ground-truth contemporaneous and lagged adjacency matrices. Topology, mechanism, noise, observational mask, and intervention policy are independent components that compose through configuration alone. Paired counterfactual twins, produced by caching the stochastic realization during the factual rollout and re-injecting it during the interventional rollout, recover individual treatment effects to numerical precision. A reference benchmark of eight discovery algorithms on eighty-one configurations and four treatment-effect estimators on eighteen configurations exposes failure modes that aggregate metrics hide. These include the inability of standard discovery methods to handle missing values, regardless of whether the missingness mechanism is informative, and a marginal-variance shortcut (varsortability) that we measure and remove by standardization.

1 Introduction

The discovery of causal relationships from observational multivariate time series is a foundational prerequisite for understanding complex dynamic systems in disciplines spanning healthcare [1], algorithmic finance, and climate science [2, 3]. Over the past decade, researchers have developed numerous algorithms for time-series causal discovery, including Granger-style methods, constraint-based independence testing, score-based continuous optimization, and deep functional approaches.

Despite this methodological proliferation, empirical evaluation has lagged. Existing benchmarks fall into two architectural patterns. The first is the *static corpus*: a fixed collection of pre-generated datasets shipped alongside a paper or maintained as a community repository [4]. The second is the *scenario script collection*: a set of generator programs, each hardcoding a specific combination of

[1] Code and data: https://github.com/hferdous/Composable_Causality.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

mechanism, confounding pattern, and missingness assumption [5]. Both patterns share a common limitation: researchers cannot ablate difficulty axes or run controlled sensitivity sweeps without authoring new generator code per configuration.

We argue this is the wrong abstraction for methodological research. The questions that drive method development (*at what sparsity does constraint-based discovery break? how does method ranking change as nonlinearity intensifies? what is the marginal cost of MCAR vs MNAR under otherwise identical conditions?*) require factorial experiments in which one data-generating process (DGP) axis varies while all others are held fixed. Static corpora and per-scenario scripts do not support this workflow.

Contributions. We introduce DTCD, a configurable framework for temporal benchmarking that replaces the fixed-artifact paradigm.

1. **A composable framework rather than a corpus or script collection.** Every DGP component (topology generator, structural mechanism, noise distribution, observational mask, intervention policy) is a first-class API parameter that composes orthogonally with every other component. Investigations that require new code in script-based benchmarks (sensitivity sweeps, factorial ablations, controlled perturbation studies) require only configuration changes in DTCD.
2. **Exact counterfactual twin generation.** The `CounterfactualSuite` caches the stochastic realization $\mathbf{N}$ during the factual rollout and re-injects it during the interventional rollout, so the individual treatment effect at any time t for any downstream node is exact to floating-point precision. No prior temporal benchmark provides ground-truth counterfactuals.
3. **Explicit identifiability documentation.** Identifiability status (full DAG, Markov equivalent, or not established) is documented for the linear mechanisms, as propositions, and for the recursive nonlinear mechanism, which is used only as a robustness stressor. No claim is made for the remaining mechanisms. Machine-readable tags currently exist only in the spatio-temporal module; adding them to the core generators is planned.
4. **A reference benchmark and diagnostic protocol.** We evaluate eight causal discovery algorithms and four CATE estimators across a tractable factorial grid of nine predefined graphs ($d \in \{4, 6, 8\}, \tau_{\max} \in \{2, 3, 4\}$) under nine discovery DGP variants (linear, nonlinear, drift, MNAR, MCAR, trend, seasonal, trend+seasonal, nonlinear+trend+seasonal) and two CATE variants. Tuning uses Optuna with standardized metrics (SHD, normalized SHD, F1, TPR, FDR, AUPRC, PEHE), and recovered graphs are visualized beside ground truth to surface failure modes aggregate metrics conceal.
5. **Open-source release with reproducibility infrastructure.** Native exporters to `tigramite` and `causal-learn`, Datasheet for Datasets [6] documentation, Croissant [7] metadata, single-command reproduction of every paper table and figure.

2 Related work

Time-series causal discovery methods. Time-series causal discovery comprises four broad paradigms. Granger-style methods extend classical linear prediction tests [8], and VAR-LiNGAM [9] provides identifiability under non-Gaussian noise. Constraint-based methods evaluate conditional independence on lagged variables. PCMCI [10] handles high-dimensional autocorrelated data and PCMCI+ [11] adds contemporaneous links, while LPCMCI [12] relaxes causal sufficiency and CD-NOD [13] exploits non-stationarity as an identifiability resource. Recent work targets non-stationary regimes directly. FANTOM [14] jointly infers unknown regimes and a DAG per regime under non-Gaussian or heteroscedastic noise, CDANs [1, 15] identifies lagged and contemporaneous edges together with the variables whose mechanisms change over time, DCD [16] targets autocorrelated and non-stationary data through a decomposition-based approach, and mixture causal discovery [17] recovers one graph per latent component when each series comes from one of several structural causal models. DTCD can emulate these settings by composition (parent-dependent or drifting noise scales, scheduled switches between adjacency structures, random-walk edge weights, per-series mixtures of configurations); benchmarking methods built for them is left to future work. Score-based continuous optimization frameworks adapted from NOTEARS [18] include DYNOTEARS [19], the nonparametric NTS-NOTEARS [20], and the deep flow-based Rhino [21]. Neural functional approaches include

attention-based TCDF [22], amortized causal discovery [23], and component-wise MLP and LSTM models for nonlinear Granger causality [24]. CUTS+ [25] addresses high-dimensional time series with missing data via joint neural imputation and discovery.

Static benchmarks and dataset corpora. The CauseMe platform [4] hosts time series with known ground truth from synthetic models that mimic real-data challenges and serves as a common reference for comparing methods. The DREAM challenges [26, 27], designed for gene-regulatory network inference, are often repurposed as time-series benchmarks despite their biology-specific generative process. NetSim [28] provides fixed simulated datasets for evaluating fMRI connectivity methods. Each of these resources fixes the data at release, so researchers cannot ablate difficulty axes without writing new generation code.

Script-based and configurable benchmark generators. TimeGraph [5] expanded coverage with scenario-specific generators, each pairing a linear, polynomial, or trigonometric mechanism with fixed choices of trend, seasonality, noise, sampling, missingness, and confounding, so combining one scenario's mechanism with another's noise or missingness requires modifying generator code. Lawrence et al. [29] propose a flexible data generating process that lets users violate common assumptions and measure how methods degrade. SynTReN [30] simulates steady-state gene expression data with hardcoded mechanisms. The `tigramite` package [10] includes a parameterized generator of structural causal processes and `causal-learn` [31] provides discovery algorithms and evaluation utilities, but neither composes generation, masking, intervention, and evaluation into benchmark suites. Peters et al. [32] formalized the distinction between full time graphs and summary graphs that time-series benchmarks, including ours, inherit. DTCD keeps the full time graph, replaces per-scenario scripts with one compositional API that also covers observation masks, interventions, and exact counterfactual twins, and exports to `tigramite` and `causal-learn` formats.

Treatment-effect estimation from time series. Synthetic control [33] estimates the effect of an intervention on one treated unit from a weighted combination of untreated units, relaxing the parallel trends assumption. Recurrent marginal structural networks [34] use recurrent networks to estimate inverse probability of treatment weights for time-dependent confounding, and the Counterfactual Recurrent Network (CRN) [35] instead learns balanced representations through adversarial training. Representation-learning CATE estimators for static data include TARNet [36], SNet [37], and FlexTENet [38], while G-Net [39] and the Causal Transformer [40] estimate counterfactual outcomes under time-varying treatments. Evaluation in this literature has typically relied on per-paper simulators or on conditional expectations rather than exact individual treatment effects. DTCD supports a standardized PEHE on exact counterfactual twins, which requires cached-noise generation. CRN, G-Net, and the Causal Transformer are not yet in our reference CATE benchmark because each needs a wrapper exposing a unified `fit(X, A, mask) -> tau_hat` interface; this is planned follow-up work.

Concurrent work on temporal interventional and counterfactual data. Two recent works address adjacent but distinct problems. Thumm and Chen [41] produce paired observational and interventional time series sampled from temporal structural causal models, with the goal of training prior-data fitted networks for in-context causal effect estimation. Their generated data serves as a training distribution for foundation models, not as a benchmark for evaluating discovery or estimation methods; the framework provides interventional rollouts but does not document identifiability status per mechanism, ship reference benchmark scripts, or expose discovery-method exporters. Xia et al. [42] introduce a diffusion-based generative model that produces interventional time series via backdoor adjustment and counterfactual time series via abduction, action, and prediction, so its counterfactuals come from a learned model rather than a known structural equation. Their counterfactual quality is therefore subject to model fidelity, whereas DTCD's counterfactual twins are exact by construction. DTCD is complementary to both: where Thumm and Chen [41] provide a training distribution and Xia et al. [42] provide a generative model, DTCD provides a benchmark library with deterministic ground truth and documented identifiability status.

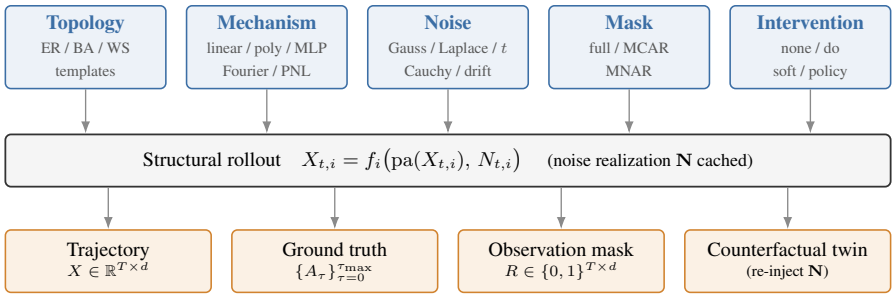


Figure 1: Compositional structure of DTCD. Five independent configuration axes feed one structural rollout, which emits the trajectory, the ground-truth adjacencies, the observation mask, and, when an intervention policy is supplied, an exactly paired counterfactual twin via re-injection of the cached noise $\mathbf{N}$. Varying one axis holds all others fixed by construction.

3 Framework design

3.1 Data-generating process

A temporal causal model over d variables evolves according to structural equations:

$$X_{t,i} = f_i\left(\mathrm{pa}(X_{t,i}), N_{t,i}\right), \tag{1}$$

where $\mathrm{pa}(X_{t,i})$ comprises both contemporaneous parents $X_{t,j} : (j \to i) \in \mathcal{A}_0$ and lagged parents $X_{t-\tau,j} : (j \to i, \tau) \in \mathcal{A}_\tau$ for $\tau \in \{1, \ldots, \tau_{\max}\}$, with $\tau_{\max}$ the maximum admissible lag for dependencies. The stochastic noise $N_{t,i}$ is independent across i (causal sufficiency assumption) but may be autocorrelated within i (supported via AR noise generators).

DTCD decomposes DGP specification into orthogonal configuration axes (Figure 1):

Graph topology. Topology generators accept $d, \tau_{\max}, \text{sparsity} \in [0, 1]$ and return a binary adjacency tensor $\mathcal{A} \in \{0,1\}^{d\times d\times(\tau_{\max}+1)}$, where $\mathcal{A}_{i,j,\tau} = 1$ indicates $X_{t-\tau,j} \to X_{t,i}$. Supported generators include Erdős-Rényi, scale-free Barabási-Albert, and small-world Watts-Strogatz graphs, plus the nine curated templates of the reference benchmark (Appendix A).

Structural mechanism. The functional form f_i is linear, polynomial, tanh, pairwise-interaction, Fourier-basis, or a multilayer perceptron, optionally wrapped in a strictly monotone post-nonlinear distortion or combined multiplicatively with the noise; softmax and Poisson mechanisms cover discrete variables. Section 3.2 documents the identifiability status of the linear mechanisms and of the recursive nonlinear one; no identifiability claim is made for the other mechanisms. Linear Gaussian mechanisms identify only up to Markov equivalence; linear non-Gaussian mechanisms (VAR-LiNGAM-compatible) identify the full DAG; the nonlinear power-transform mechanism is not provably identifiable in its recursive form and is provided as a robustness stressor rather than an identifiability target.

Noise distribution. Supported noise models are Gaussian, Laplace, Student-t, exponential, uniform, Cauchy, heteroscedastic, drifting-scale, and mixtures. Non-Gaussian noise is required for VAR-LiNGAM identifiability. Noise realizations are cached per timestep and variable during generation, enabling exact counterfactual queries (Section 3.3).

Observational mask. Missing data patterns follow MCAR and MNAR mechanisms. The mask $R \in \{0,1\}^{T\times d}$ is generated jointly with X. Under MCAR, each entry is masked with probability p independent of the value; under MNAR, masking is implemented via self-thresholding (each variable's top-quantile values are masked). Including both mechanisms enables controlled isolation of missing-value robustness from informativeness of the missingness pattern.

Exogenous components. Optional additive trend (linear in time) and seasonal (sum of sinusoids at configurable periods, supporting both single and multi-period superpositions such as daily $\times$ weekly

$\times$ annual cycles) components can be applied per variable as post-hoc additions to the structural output:

$$X_{t,i}^{\text{obs}} = X_{t,i} + \alpha_i \cdot t/T + \sum_k \beta_{i,k} \sin(2\pi t/P_k + \phi_{i,k}).$$

These do not enter the structural equations and the ground-truth graph is unchanged across variants with or without them. Their inclusion probes method robustness to non-stationary observational artifacts that are common in real-world deployments but absent from canonical synthetic benchmarks.

Intervention policy. The CounterfactualSuite accepts $\text{do}(X_{\tau,i} = c)$ interventions at specified timesteps. Downstream effects propagate through the graph via the cached noise $\mathbf{N}$. This construction yields exact individual treatment effects: $\text{ITE}_{t,j} = X_{t,j}^{\text{do}} - X_{t,j}^{\text{obs}}$ is floating-point exact because the only difference between factual and counterfactual rollouts is the intervened node value at the intervention timestep.

3.2 Identifiability guarantees

We document the identifiability class of the linear mechanisms as follows:

Proposition 1 (Linear Gaussian VAR identifiability)**.** *Suppose the data are generated by a stationary linear vector autoregression with: (i) causal sufficiency (no latent confounders), (ii) Gaussian innovations $N_{t,i} \sim \mathcal{N}(0, \sigma_i^2)$ with $\sigma_i > 0$, (iii) the contemporaneous adjacency matrix A_0 acyclic, and (iv) faithfulness (Markov properties imply structural ones). Then the supports of the lagged adjacency matrices $\{A_\tau\}_{\tau=1}^{\tau_{\max}}$ are identified, while A_0 is identifiable only up to the Markov equivalence class of the full time graph, where lagged v-structures can orient some contemporaneous edges. This follows from reduced-form VAR identifiability [43] and the graphical characterization of Markov equivalence [44, 45].*

Proposition 2 (VAR-LiNGAM identifiability)**.** *Suppose the data are generated by a stationary linear vector autoregression with: (i) causal sufficiency, (ii) non-Gaussian, mutually independent innovations $N_{t,i}$ with at most one Gaussian-distributed component, (iii) acyclicity at each time slice, and (iv) the contemporaneous adjacency matrix A_0 can be permuted to a strictly lower-triangular matrix (LiNGAM acyclicity condition). Then both A_0 and $\{A_\tau\}_{\tau=1}^{\tau_{\max}}$ are uniquely identified [9]. The Gaussian-noise restriction in Proposition 1 is replaced by non-Gaussianity, which restores full DAG identifiability via independent component analysis on the residuals.*

Nonlinear power-transform (recursive form). The library's nonlinear variant applies a sign-preserving saturating distortion $X_i \leftarrow \text{sign}(Y_i)\,|Y_i|^{0.5}$ where $Y_i = \sum_j W_{ji} X_j + \sum_{\tau,j} (A_\tau)_{ji} X_{t-\tau,j} + N_{t,i}$. Because the transformed value is read by downstream variables as a parent in the same time slice, the resulting model is a recursive composition of distortions rather than a single-application post-nonlinear model. We do not establish identifiability for this recursive form; it is included to test method robustness to non-Gaussian, non-additive structure, not as an identifiability benchmark. We treat configurations using this mechanism as identifiability-unknown; the released core generators do not attach a machine-readable tag to them. Users requiring provably identifiable nonlinear DGPs should instead select the single-application post-nonlinear or additive-noise mechanisms, both of which are also supported.

3.3 Counterfactual twin generation

Traditional causal effect estimation benchmarks provide separate datasets for treatment and control groups, making individual treatment effect (ITE) evaluation impossible. DTCD resolves this via cached-noise rollouts:

- **Factual generation:** For each (t, i), generate $N_{t,i} \sim \mathcal{D}_i$, store it, and compute $X_{t,i} = f_i(\text{pa}(X_{t,i}), N_{t,i})$.
- **Intervention:** Specify $\text{do}(X_{\tau,i} = c)$ at intervention time τ.
- **Counterfactual rollout:** Re-execute the forward pass from τ onward, using the *cached* noise $\mathbf{N}$ but replacing $X_{\tau,i}$ with c.

Because the stochastic component is held fixed, $\text{ITE}_{t,j}$ is exact. This enables evaluation metrics that require ground-truth individual effects: PEHE (Precision in Estimation of Heterogeneous Effects) and ϵ-ATE (average treatment effect error).

4 Experimental protocol

4.1 Benchmark configuration

We instantiate a tractable factorial design spanning fundamental difficulty axes:

Graph configurations. Nine templates: $(d, \tau_{\max}) \in \{4, 6, 8\} \times \{2, 3, 4\}$. Each graph is hand-curated to include contemporaneous edges, lagged edges across several lags, and collider structures; all nine are acyclic. Sparsity is calibrated to maintain $\approx d$ edges per graph to avoid trivial or saturated regimes.

DGP variants. Figure 2 in Appendix A shows one graph rendered under four variants.

- **Linear:** VAR with Gaussian noise. Identifies only contemporaneous CPDAG.
- **Nonlinear:** Linear VAR followed by recursive sign-preserving power transform $\text{sign}(\cdot)|\cdot|^{0.5}$. Identifiability not established for the recursive form.
- **Drift:** Linear VAR with linearly time-varying coefficients. Tests robustness to non-stationarity.
- **MNAR:** Linear VAR with self-thresholded missingness (top quantile of each variable masked). Tests methods on informative missingness.
- **MCAR:** Linear VAR with uniform random missingness (20% per entry, independent of values). Serves as controlled comparator to MNAR, isolating the cost of informativeness from the cost of missingness itself.
- **Trend, Seasonal, Trend+Seasonal:** Linear VAR augmented with additive exogenous deterministic components. The structural graph is unchanged; these variants probe sensitivity to non-stationary observational artifacts that do not enter the structural equations.
- **Nonlinear+Trend+Seasonal:** Combination of the nonlinear mechanism above with additive trend and seasonality.

Hyperparameter tuning. Every method with tunable hyperparameters undergoes Optuna [46] TPE optimization with a budget of 30 trials per (method, cell) combination (VARLiNGAM and VARLiNGAM_imp have no tunable parameters), optimized on oriented F1 against the ground-truth graph. Tuning is performed on a dedicated trajectory at seed 41 that is held out of the reported evaluation set; no evaluation seed is ever seen by the tuning objective. Selected hyperparameters are cached at the (method, graph, variant) level and reused across all evaluation seeds for that cell, isolating tuning from the variance attributed to noise realizations. Two methods with prohibitive runtime, PCMCI-GPDC and DYNOTEARS, are tuned with a reduced budget of 5 trials. We note that the tuning objective is an oracle: it uses the ground-truth graph, which is unavailable in application. Reported numbers for tuned methods are therefore optimistic relative to what is attainable without the graph.

The benchmark fixes trajectory length $T{=}1000$, standard Gaussian innovations $\varepsilon_{t,i} \sim \mathcal{N}(0, 1)$ shared across all variables, structural weights drawn from $\mathcal{U}(0.3, 0.8)$ with random sign and rejection-sampled to spectral radius $\rho < 0.95$, sparsity calibrated to $\approx d$ edges per graph, tuning seed 41 and evaluation seeds $\{42, \ldots, 51\}$. Variant-specific parameters (drift slope, nonlinear transform, trend/seasonal activation, missingness fractions, CATE assignment policy) are tabulated in full in Appendix A.

Evaluation protocol. For each of the nine predefined graphs, the structural weights W_0 and $\{A_\tau\}$ are sampled once using a graph-derived weight seed and held fixed across all evaluation seeds, all DGP variants, and all methods. The ten evaluation seeds (42–51) vary only the noise realization $\{N_{t,i}\}$ within each fixed (graph, variant) cell. Reported standard deviations therefore reflect noise variance under held-fixed weights, not weight variance under held-fixed noise. This protocol is a deliberate design choice: it isolates method sensitivity to the stochastic component from confounding due to weight realization. A complementary weight-sensitivity analysis varying weights at fixed noise is supported by the framework but deferred to future work.

Table 1: Causal discovery performance on Linear and Nonlinear DGP variants. Mean ($\pm$ std) over 90 runs per cell (9 graphs $\times$ 10 seeds), filtered to `status=ok` rows. Lower SHD and FDR are better; higher TPR is better. Bold marks the best value in each column. PCMCI-GPDC values reflect d=4 cells only (n=30 versus n=90 for other methods). Performance on Drift is within 0.08 SHD of Linear for every method; full results in Appendix H.

	Linear			Nonlinear		
	SHD	TPR	FDR	SHD	TPR	FDR
DYNOTEARS	**0.02 ± 0.05**	**1.00 ± 0.02**	**0.01 ± 0.04**	**0.02 ± 0.05**	**0.99 ± 0.03**	**0.01 ± 0.03**
PCMCI_GPDC	0.38 ± 0.17	0.67 ± 0.12	0.05 ± 0.11	0.41 ± 0.15	0.67 ± 0.12	0.07 ± 0.13
VARLiNGAM	0.52 ± 0.40	0.80 ± 0.17	0.27 ± 0.20	0.50 ± 0.41	0.79 ± 0.22	0.25 ± 0.20
VARLiNGAM_imp	0.52 ± 0.40	0.80 ± 0.17	0.27 ± 0.20	0.50 ± 0.41	0.79 ± 0.22	0.25 ± 0.20
PCMCI_ParCorr	0.45 ± 0.14	0.65 ± 0.07	0.12 ± 0.13	0.42 ± 0.12	0.65 ± 0.07	0.08 ± 0.10
CUTS+	0.48 ± 0.24	0.87 ± 0.31	0.36 ± 0.24	0.44 ± 0.17	0.94 ± 0.12	0.27 ± 0.09
GrangerF	1.83 ± 0.58	0.65 ± 0.07	0.67 ± 0.09	1.83 ± 0.59	0.65 ± 0.07	0.67 ± 0.10
CDNOD	1.03 ± 0.10	0.03 ± 0.07	0.92 ± 0.17	1.02 ± 0.07	0.02 ± 0.05	0.94 ± 0.16

Table 2: Causal discovery performance on missingness DGP variants. Only the two methods declaring `supports_missing=True` are shown; the remaining six methods were skipped on these variants (see Appendix I). Bold marks the best value in each column.

	MNAR			MCAR		
	SHD	TPR	FDR	SHD	TPR	FDR
VARLiNGAM_imp	0.96 ± 0.37	0.52 ± 0.26	0.47 ± 0.23	0.79 ± 0.43	0.64 ± 0.26	0.40 ± 0.24
CUTS+	**0.94 ± 0.45**	**0.73 ± 0.18**	**0.42 ± 0.16**	**0.67 ± 0.43**	**0.94 ± 0.10**	**0.35 ± 0.13**

Baselines. We evaluate eight causal discovery methods spanning the four established paradigm classes (Granger-style, constraint-based, score-based, functional) and four CATE estimators covering representation-learning, structured-information, and meta-learning families. Discovery methods comprise VARLiNGAM [9] and its mean-imputation variant VARLiNGAM_imp, PCMCI [10] with both ParCorr and GPDC conditional independence tests, DYNOTEARS [19], CD-NOD [13], CUTS+ [25], and a linear F-test Granger [8] baseline (GrangerF). CATE estimators comprise TARNet [36], FlexTENet [38], SNet [37], and X-Learner [47].

A unified wrapper interface (`fit(X, mask)` with declared `supports_missing` flags) lets us treat heterogeneous methods uniformly: when a method declares it cannot accept missing inputs, the runner records a structured `skipped_no_missing_support` status on MCAR and MNAR cells rather than silently emitting an empty graph. PCMCI-GPDC's GP-kernel cost restricts it to d=4 cells, with $d \in \{6, 8\}$ reported as `skipped_compute_budget`; CD-NOD targets contemporaneous causal structure under non-stationarity and does not detect lagged edges, which directly explains the low recall reported in Table 1. Per-method details appear in Appendix B.

Metrics. *Discovery:* Structural Hamming Distance (SHD), normalized SHD (SHD / max possible edges), skeleton F1, orientation F1, true-positive rate (TPR), false-discovery rate (FDR), precision, recall, and AUPRC. *CATE:* PEHE (Precision in Estimation of Heterogeneous Effects) and runtime.

4.2 Causal discovery results

Tables 1 and 2 present aggregate performance across observational and missingness DGP variants respectively. The ranking is strikingly DGP-dependent for some method classes and remarkably stable for others, a contrast we explore below.

Key findings.

DYNOTEARS dominates clean conditions with F1 ≈ 0.99 on Linear and Nonlinear and 0.98 on Drift (Appendix H; SHD 0.02, 0.02 and 0.04). Recovery on the Nonlinear variant is driven by the recursive sign-power transform's strict monotonicity and bounded amplitude rather than general nonlinear robustness, as we examine in Section 5 and Appendix G. DYNOTEARS does not support missing inputs and is therefore absent from MCAR and MNAR.

CUTS+ is the strongest method on missing data (F1 = 0.75 on MCAR, 0.62 on MNAR), completing all 90 cells under each and beating VARLiNGAM_imp on SHD, TPR, and FDR under both. The 0.14-

point gap quantifies the cost of informativeness for a method that handles missingness structurally rather than through input rejection.

Linear conditional-independence methods are insensitive to monotonic and additive perturbations. At evaluation seed 42, PCMCI-ParCorr and GrangerF return the same SHD and predicted-edge count on the Linear and Nonlinear versions of a graph in 15 of 18 (graph, method) pairs.

VARLiNGAM_imp completes every cell (F1 = 0.61 on MCAR, 0.50 on MNAR) after a wrapper fix; the 171 failures of our first run biased its earlier scores upward (Appendix I).

CD-NOD's near-zero recall (TPR 0.02 to 0.03) reflects CD-NOD's design as a contemporaneous-structure recovery method, not a method failure. **GrangerF serves as the adversarial floor** (F1 ≈ 0.43, FDR ≈ 0.67).

Per-dimension performance breakdowns appear in Appendix F: DYNOTEARS scales favorably (SHD essentially flat across $d \in \{4, 6, 8\}$), while PCMCI-ParCorr and GrangerF degrade at $d = 8$; PCMCI-GPDC's GP-kernel cost places $d \in \{6, 8\}$ outside our compute budget.

Failure modes and method-level instability. Of 6,480 attempted (method, graph, variant, seed) combinations, 4,809 succeeded; 960 were skipped because the method declared `supports_missing=False` on a missingness variant; 540 were skipped on PCMCI-GPDC at $d > 4$ by the compute-budget guard; and none raised an exception. Our first run had 171 exceptions (2.6%), all in VARLiNGAM_imp, from a wrapper defect; this paper reports a full regeneration with the wrapper fixed (Appendix I).

4.3 Varsortability and the marginal-variance shortcut

Additive-noise generators with homoscedastic innovations and coefficients bounded away from zero accumulate variance along structural paths. Reisach et al. [48] show that the resulting *varsortability* v, the fraction of directed paths running from lower to higher marginal variance, can make a benchmark solvable by sorting variables by variance and regressing, without causal reasoning. We measure v rather than assume it away.

Table 5 and Figure 5 in Appendix A report the audit per graph; the headline numbers follow. Contemporaneous varsortability on the raw benchmark averages 0.93 (0.5 is uninformative, 1.0 perfectly sortable), so we add two controls. SORTNREGRESS orders contemporaneous variables by increasing marginal variance and fits an adaptive lasso; RANDOMREGRESS is the identical estimator with a random order; lagged variables are candidate parents for both, since time resolves their direction. On raw data SORTNREGRESS reaches $F_1 = 0.93$ against DYNOTEARS at 0.99, so most of the linear variant's apparent difficulty is carried by marginal variance. Standardizing drives v to exactly 0.5 while leaving the graph unchanged; DYNOTEARS falls to 0.82, SORTNREGRESS converges to RANDOMREGRESS (0.81 against 0.79, the latter invariant across conditions), the margin of DYNOTEARS over the best competing method shrinks from 0.23 to 0.05, and re-tuning separately for the standardized condition on the held-out trajectory recovers less than 0.01. Across the nine DGP variants the effect is material (> 0.05) on six, largest on seasonal (0.79 to 0.46), absent on the trend variants whose exogenous variance dilutes v, and exactly zero for every scale-invariant method (PCMCI-ParCorr, VARLiNGAM, GrangerF), so the artifact is method-selective. Figure 4 in Appendix A shows the effect on one cell, F_1 1.00 raw and 0.44 standardized.

Two conclusions follow. First, the near-perfect linear-variant scores in Table 1 overstate what these methods recover from structure alone, with the score-based method affected most, consistent with Reisach et al. [48]. Second, the effect is bounded rather than total: the contemporaneous sub-graphs are shallow, so one to three of the four to eight edges per graph are exposed to the shortcut, and standardized performance remains well above chance. DTCD now ships standardization as a first-class DGP axis (the `_std` variants), records v in the metadata of every generated cell, and includes both controls in the default baseline set, so any result can be read against the shortcut it might be exploiting. Appendix K applies the same instrument to an independently authored equation family and finds the reverse regime, high difficulty with no shortcut.

4.4 Counterfactual estimation results

Table 3 presents CATE estimator performance. The twin generation protocol enables PEHE evaluation against ground-truth individual treatment effects, a measurement unavailable in conventional benchmarks that compare predictions to conditional expectations.

Table 3: CATE estimator performance aggregated by DGP variant. Mean ($\pm$ std) over 90 runs per cell (9 graphs $\times$ 10 seeds). PEHE: Precision in Estimation of Heterogeneous Effects; lower is better. Runtime is wall-clock per fit, measured under four parallel processes (Appendix L); compare ratios only.

	CATE Linear		CATE Nonlinear	
	PEHE	Runtime (s)	PEHE	Runtime (s)
TARNet	$\mathbf{0.491 \pm 0.173}$	1.639 ± 0.349	$\mathbf{0.427 \pm 0.135}$	1.639 ± 0.351
SNet	0.674 ± 0.220	11.571 ± 1.436	0.509 ± 0.129	12.444 ± 1.988
FlexTENet	0.663 ± 0.201	6.385 ± 1.244	0.518 ± 0.129	6.636 ± 1.302
X-Learner	0.757 ± 0.155	1.380 ± 0.078	0.617 ± 0.129	1.374 ± 0.078

Key findings. TARNet wins on both variants (PEHE = 0.49 Linear, 0.43 Nonlinear) with a runtime of about 1.6 seconds, comparable only to X-Learner; the representation-balancing objective fits the heterogeneous effect structure DTCD produces under randomized assignment. **All methods improve from Linear to Nonlinear** (PEHE drops 13–25% across all four estimators). The pattern reflects the saturating sign-power transform compressing the range of treatment-effect modifiers, narrowing effective heterogeneity, a mechanism-level interaction that aggregate single-DGP benchmarks would not surface. **Runtime-accuracy tradeoffs are stark**: TARNet and X-Learner complete in under 2 seconds; FlexTENet takes about 6.5 seconds and trails TARNet by 0.09 to 0.17 PEHE; SNet's 12 seconds buys no PEHE advantage over FlexTENet, which may reflect a mismatch between its orthogonal-information design and DTCD's relatively short ($T = 1000$) sequences. **Method ranking is preserved across variants** (TARNet $<$ SNet $\approx$ FlexTENet $<$ X-Learner on both); deeper architectures provide little additional benefit once the average effect is captured.

Per-dimension PEHE patterns on the Linear DGP (Appendix F) are mixed: X-Learner's PEHE falls with d, TARNet's does not rise monotonically, and SNet's and FlexTENet's rise.

5 Discussion

Inspecting individual failures reveals patterns hidden by aggregate metrics. CD-NOD's TPR of 0.02 to 0.03 reflects the algorithm's design: CD-NOD recovers contemporaneous causal structure under non-stationarity and does not detect lagged edges, so when evaluated against ground-truth graphs that contain both, its measured recall undercounts what the method is designed to find. This is a benchmark-versus-method scope mismatch rather than a method failure, and we report it transparently. GrangerF's FDR ≈ 0.67 is consistent with Granger causality's known inability to distinguish direct from mediated effects. PCMCI-ParCorr and GrangerF return unchanged SHD and edge counts between Linear and Nonlinear in most graphs (Section 4), consistent with linear conditional-independence tests being insensitive to monotonic distortion.

DYNOTEARS' nonlinear-DGP performance reflects monotonicity plus signal-amplitude stability. Although DYNOTEARS optimizes a linear SEM, it achieves near-perfect F1 on the Nonlinear variant. The recursive sign-power transform $\text{sign}(x)|x|^{0.5}$ is strictly monotonic and bounded by the square-root saturation, so the sign of every partial derivative is preserved and signal magnitudes remain well-conditioned for linear regression. We probe this boundary empirically in Appendix G: a non-monotonic alternative ($\sin x$) drops F1 from 0.99 to 0.88 and raises mean SHD from 0.11 to 1.14. The Nonlinear variant remains identifiability-unknown (Section 3.2); we report this as bounded robustness evidence under monotonic distortion rather than identifiability evidence.

Missing values defeat methods regardless of mechanism. Six of eight discovery methods cannot accept NaN inputs at all. Of the two that opt in, CUTS+ completes all 90 cells under both MCAR and MNAR (F1 0.75 and 0.62). We attribute the 0.14-point gap, without having tested it, to neural imputation extrapolating poorly into MNAR's systematically masked upper quantiles. VARLiNGAM_imp also completes every cell (F1 0.61 and 0.50) and trails CUTS+ on both; its earlier failures were a wrapper defect, not missingness (Appendix I). What the benchmark does show is that input-validation barriers, not identifiability, keep most methods from running on missing data at all; DTCD's MCAR variant exists to make that distinction observable.

Counterfactual evaluation reveals architectural mismatches. Method ranking is ordinally stable across DGP variants (TARNet $<$ SNet $\approx$ FlexTENet $<$ X-Learner on both Linear and Nonlinear), in

contrast to the discovery side where DGP-dependence reshuffled rankings. Architectural complexity does not translate to PEHE gains: SNet's orthogonal-information design takes about seven times TARNet's runtime to produce worse PEHE on both variants, a mismatch flat ATE-error reporting would hide.

Dimensionality is not uniformly detrimental. On the Linear DGP, X-Learner's PEHE falls with d (0.84, 0.82, 0.61 for $d = 4, 6, 8$) TARNet's does not rise monotonically (0.44, 0.59, 0.44), and SNet's and FlexTENet's rise (Appendix F). We have not tested why, and with three dimensions and 30 runs per cell we treat it as a pattern to examine. This underscores why compositional frameworks that isolate the causal effect of each axis are valuable.

6 Limitations and broader impact

6.1 Limitations

What synthetic data can and cannot establish. A synthetic generator fixes the mechanism class by construction, and that class tends to resemble the model forms prevailing when the generator is written, so results on DTCD are evidence about behavior under a stated DGP, not about empirical data; Appendix K quantifies this family dependence on an independently authored DGP. Synthetic evaluation is nonetheless required rather than optional. Structure recovery is scored against the true graph, which no observational corpus supplies and expert annotation only approximates, and the counterfactual outcomes needed to score treatment-effect estimation exactly are never observed. The right reading is therefore diagnostic rather than confirmatory, one leg of an evaluation that also includes real data, and we provide export adapters to `tigramite` and `causal-learn` for that purpose. Section 4.3 is a worked instance of the risk, a generator artifact that no aggregate metric surfaces, found only by measuring for it directly; we expect others remain, and we ship the controls partly so the next one is easier to catch.

Scope. The reference benchmark spans $d \in \{4, 6, 8\}$, with PCMCI-GPDC restricted to $d{=}4$ on compute grounds; larger graphs (Appendix J) and the spatio-temporal module are supported but outside the reference benchmark. Deferred baselines and implementation notes are in Appendix B.4.

Identifiability. Configurations whose identifiability is not established (recursive nonlinear power-transform; non-monotonic post-nonlinear distortions; simultaneous MNAR on confounded variables; feedback without lag separation) are treated as identifiability-unknown. Only the spatio-temporal module currently emits identifiability tags and construction-time warnings; adding them to the core generators is planned. Performance on these configurations is robustness evidence, not identifiability evidence.

Protocol. Standard deviations capture noise-realization variance under weights held fixed per graph; weight-realization sensitivity is supported but deferred. Hyperparameter selection uses an oracle objective (F1 against the ground-truth graph) on a held-out trajectory, so reported figures are optimistic relative to what is attainable without the graph. CATE assignment is Bernoulli(0.5), isolating function approximation from deconfounding; confounded-assignment variants are scheduled for the next release. The `CounterfactualSuite` presumes causal sufficiency of the simulator's variable set, which holds by construction but does not transfer to empirical counterfactual evaluation. The reference results are deterministic given their seeds on one machine, which we checked by rerunning selected cells in separate processes (Appendix I); we have not tested other machines.

6.2 Broader impact

DTCD is a research benchmark; it does not deploy decisions, control physical systems, or process personal data, and we see no misuse pathway specific to it. Explicit identifiability documentation should reduce methods being credited for performance on unidentifiable configurations, and protocol-level standardization (uniform tuning, structured failure reporting, controlled DGP variation) improves rigor in follow-up work. A Datasheet (Appendix N) and Croissant metadata accompany the artifact, under Apache-2.0 (library) and CC-BY-4.0 (datasets).

Acknowledgments and Disclosure of Funding

We thank the NeurIPS reviewers and the area chair, whose questions during the discussion period led directly to the varsortability audit of Section 4.3, the scalability study of Appendix J, and the corrected tuning protocol. Experiment execution and manuscript editing were assisted by LLM-based tools, as disclosed in the submission form; the authors are responsible for all content.

Research reported in this publication was supported by the NSF iHARP: HDR Institute for Harnessing Data and Model Revolution in the Polar Regions (Award #2118285) and by the National Institute on Minority Health and Health Disparities of the National Institutes of Health (Award #R01MD019814). The views expressed in this work are solely the responsibility of the authors and do not necessarily represent the official views of the NSF or the NIH. Endorsement by the Federal Government should not be inferred.

The authors declare no competing financial interests related to this work.

References

[1] Muhammad Hasan Ferdous, Uzma Hasan, and Md Osman Gani. CDANs: Temporal causal discovery from autocorrelated and non-stationary time series data. In *Proceedings of the 8th Machine Learning for Healthcare Conference*, volume 219 of *Proceedings of Machine Learning Research*, pages 186–207. PMLR, 2023.

[2] Emam Hossain, Muhammad Hasan Ferdous, Jianwu Wang, Aneesh Subramanian, and Md Osman Gani. Correlation to causation: A causal deep learning framework for Arctic sea ice prediction. In *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events (PerCom Workshops)*, pages 62–67. IEEE Computer Society, 2025.

[3] Emam Hossain, Muhammad Hasan Ferdous, Devon Dunmire, Aneesh Subramanian, and Md Osman Gani. Causal time series modeling of supraglacial lake evolution in Greenland under distribution shift. In *2025 International Conference on Machine Learning and Applications (ICMLA)*, 2025.

[4] Jakob Runge, Sebastian Bathiany, Erik Bollt, Gustau Camps-Valls, Dim Coumou, Ethan Deyle, Clark Glymour, Marlene Kretschmer, Miguel D. Mahecha, Jordi Muñoz-Marí, Egbert H. van Nes, Jonas Peters, Rick Quax, Markus Reichstein, Marten Scheffer, Bernhard Schölkopf, Peter Spirtes, George Sugihara, Jie Sun, Kun Zhang, and Jakob Zscheischler. Inferring causation from time series in Earth system sciences. *Nature Communications*, 10:2553, 2019. doi: 10.1038/s41467-019-10105-3.

[5] Muhammad Hasan Ferdous, Emam Hossain, and Md Osman Gani. TimeGraph: Synthetic benchmark datasets for robust time-series causal discovery. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 5425–5435. ACM, 2025. doi: 10.1145/3711896.3737439.

[6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

[7] Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Luca Foschini, Pieter Gijsbers, Joan Giner-Miguelez, Sujata Goswami, Nitisha Jain, Michalis Karamousadakis, Satyapriya Krishna, Michael Kuchnik, Sylvain Lesage, Quentin Lhoest, Pierre Marcenac, Manil Maskey, Peter Mattson, Luis Oala, Hamidah Oderinwale, Pierre Ruyssen, Tim Santos, Rajat Shinde, Elena Simperl, Arjun Suresh, Goeffry Thomas, Slava Tykhonov, Joaquin Vanschoren, Susheel Varma, Jos van der Velde, Steffen Vogler, Carole-Jean Wu, and Luyao Zhang. Croissant: A metadata format for ML-ready datasets. In *Advances in Neural Information Processing Systems*, volume 37, pages 82133–82148, 2024. doi: 10.52202/079017-2610.

[8] Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969. doi: 10.2307/1912791.

[9] Aapo Hyvärinen, Kun Zhang, Shohei Shimizu, and Patrik O. Hoyer. Estimation of a structural vector autoregression model using non-Gaussianity. *Journal of Machine Learning Research*, 11 (56):1709–1731, 2010.

[10] Jakob Runge, Peer Nowack, Marlene Kretschmer, Seth Flaxman, and Dino Sejdinovic. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 5 (11):eaau4996, 2019. doi: 10.1126/sciadv.aau4996.

[11] Jakob Runge. Discovering contemporaneous and lagged causal relations in autocorrelated nonlinear time series datasets. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence*, volume 124 of *Proceedings of Machine Learning Research*, pages 1388–1397. PMLR, 2020.

[12] Andreas Gerhardus and Jakob Runge. High-recall causal discovery for autocorrelated time series with latent confounders. In *Advances in Neural Information Processing Systems*, volume 33, pages 12615–12625, 2020.

[13] Biwei Huang, Kun Zhang, Jiji Zhang, Joseph Ramsey, Ruben Sanchez-Romero, Clark Glymour, and Bernhard Schölkopf. Causal discovery from heterogeneous/nonstationary data. *Journal of Machine Learning Research*, 21(89):1–53, 2020.

[14] Abdellah Rahmani and Pascal Frossard. Flow based approach for dynamic temporal causal models with non-Gaussian or heteroscedastic noises. In *Advances in Neural Information Processing Systems*, volume 38, pages 8300–8349, 2025. doi: 10.52202/085713-0283.

[15] Muhammad Hasan Ferdous, Uzma Hasan, and Md Osman Gani. eCDANs: Efficient temporal causal discovery from autocorrelated and non-stationary data (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 16208–16209, 2023.

[16] Muhammad Hasan Ferdous, Ian Stockwell, and Md Osman Gani. DCD: Decomposition-based causal discovery from autocorrelated and non-stationary temporal data. *arXiv preprint arXiv:2602.01433*, 2026.

[17] Sumanth Varambally, Yian Ma, and Rose Yu. Discovering mixtures of structural causal models from time series data. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 49171–49202. PMLR, 2024.

[18] Xun Zheng, Bryon Aragam, Pradeep K. Ravikumar, and Eric P. Xing. DAGs with NO TEARS: Continuous optimization for structure learning. In *Advances in Neural Information Processing Systems*, volume 31, pages 9472–9483, 2018.

[19] Roxana Pamfil, Nisara Sriwattanaworachai, Shaan Desai, Philip Pilgerstorfer, Konstantinos Georgatzis, Paul Beaumont, and Bryon Aragam. DYNOTEARS: Structure learning from time-series data. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pages 1595–1605. PMLR, 2020.

[20] Xiangyu Sun, Oliver Schulte, Guiliang Liu, and Pascal Poupart. NTS-NOTEARS: Learning nonparametric DBNs with prior knowledge. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 1942–1964. PMLR, 2023.

[21] Wenbo Gong, Joel Jennings, Cheng Zhang, and Nick Pawlowski. Rhino: Deep causal temporal relationship learning with history-dependent noise. In *The Eleventh International Conference on Learning Representations*, 2023.

[22] Meike Nauta, Doina Bucur, and Christin Seifert. Causal discovery with attention-based convolutional neural networks. *Machine Learning and Knowledge Extraction*, 1(1):312–340, 2019. doi: 10.3390/make1010019.

[23] Sindy Löwe, David Madras, Richard Zemel, and Max Welling. Amortized causal discovery: Learning to infer causal graphs from time-series data. In *Proceedings of the First Conference on Causal Learning and Reasoning*, volume 177 of *Proceedings of Machine Learning Research*, pages 509–525. PMLR, 2022.

[24] Alex Tank, Ian Covert, Nicholas J. Foti, Ali Shojaie, and Emily B. Fox. Neural Granger causality. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(8):4267–4279, 2022. doi: 10.1109/TPAMI.2021.3065601.

[25] Yuxiao Cheng, Lianglong Li, Tingxiong Xiao, Zongren Li, Jinli Suo, Kunlun He, and Qionghai Dai. CUTS+: High-dimensional causal discovery from irregular time-series. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(10):11525–11533, 2024. doi: 10.1609/aaai.v38i10.29034.

[26] Robert J. Prill, Daniel Marbach, Julio Saez-Rodriguez, Peter K. Sorger, Leonidas G. Alexopoulos, Xiaowei Xue, Neil D. Clarke, Gregoire Altan-Bonnet, and Gustavo Stolovitzky. Towards a rigorous assessment of systems biology models: The DREAM3 challenges. *PLoS ONE*, 5(2): e9202, 2010. doi: 10.1371/journal.pone.0009202.

[27] Daniel Marbach, Thomas Schaffter, Claudio Mattiussi, and Dario Floreano. Generating realistic in silico gene networks for performance assessment of reverse engineering methods. *Journal of Computational Biology*, 16(2):229–239, 2009. doi: 10.1089/cmb.2008.09TT.

[28] Stephen M. Smith, Karla L. Miller, Gholamreza Salimi-Khorshidi, Matthew Webster, Christian F. Beckmann, Thomas E. Nichols, Joseph D. Ramsey, and Mark W. Woolrich. Network modelling methods for FMRI. *NeuroImage*, 54(2):875–891, 2011. doi: 10.1016/j.neuroimage.2010.08.063.

[29] Andrew R. Lawrence, Marcus Kaiser, Rui Sampaio, and Maksim Sipos. Data generating process to evaluate causal discovery techniques for time series data. *arXiv preprint arXiv:2104.08043*, 2021. Causal Discovery and Causality-Inspired Machine Learning Workshop at NeurIPS 2020.

[30] Tim Van den Bulcke, Koenraad Van Leemput, Bart Naudts, Piet van Remortel, Hongwu Ma, Alain Verschoren, Bart De Moor, and Kathleen Marchal. SynTReN: A generator of synthetic gene expression data for design and analysis of structure learning algorithms. *BMC Bioinformatics*, 7:43, 2006. doi: 10.1186/1471-2105-7-43.

[31] Yujia Zheng, Biwei Huang, Wei Chen, Joseph Ramsey, Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter Spirtes, and Kun Zhang. Causal-learn: Causal discovery in Python. *Journal of Machine Learning Research*, 25(60):1–8, 2024.

[32] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. Causal inference on time series using restricted structural equation models. In *Advances in Neural Information Processing Systems*, volume 26, pages 154–162, 2013.

[33] Alberto Abadie, Alexis Diamond, and Jens Hainmueller. Synthetic control methods for comparative case studies: Estimating the effect of California's tobacco control program. *Journal of the American Statistical Association*, 105(490):493–505, 2010. doi: 10.1198/jasa.2009.ap08746.

[34] Bryan Lim, Ahmed Alaa, and Mihaela van der Schaar. Forecasting treatment responses over time using recurrent marginal structural networks. In *Advances in Neural Information Processing Systems*, volume 31, pages 7483–7493, 2018.

[35] Ioana Bica, Ahmed M. Alaa, James Jordon, and Mihaela van der Schaar. Estimating counterfactual treatment outcomes over time through adversarially balanced representations. In *International Conference on Learning Representations*, 2020.

[36] Uri Shalit, Fredrik D. Johansson, and David Sontag. Estimating individual treatment effect: Generalization bounds and algorithms. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3076–3085. PMLR, 2017.

[37] Alicia Curth and Mihaela van der Schaar. Nonparametric estimation of heterogeneous treatment effects: From theory to learning algorithms. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pages 1810–1818. PMLR, 2021.

[38] Alicia Curth and Mihaela van der Schaar. On inductive biases for heterogeneous treatment effect estimation. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[39] Rui Li, Stephanie Hu, Mingyu Lu, Yuria Utsumi, Prithwish Chakraborty, Daby M. Sow, Piyush Madan, Jun Li, Mohamed Ghalwash, Zach Shahn, and Li-wei Lehman. G-Net: A recurrent network approach to G-computation for counterfactual prediction under a dynamic treatment regime. In *Proceedings of Machine Learning for Health*, volume 158 of *Proceedings of Machine Learning Research*, pages 282–299. PMLR, 2021.

[40] Valentyn Melnychuk, Dennis Frauen, and Stefan Feuerriegel. Causal transformer for estimating counterfactual outcomes. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 15293–15329. PMLR, 2022.

[41] Dennis Thumm and Ying Chen. Interventional time series priors for causal foundation models. *arXiv preprint arXiv:2603.11090*, 2026. ICLR 2026 Workshop on Time Series in the Age of Large Models.

[42] Yutong Xia, Chang Xu, Yuxuan Liang, Li Zhao, Qingsong Wen, Roger Zimmermann, and Jiang Bian. Causal time series generation via diffusion models. *arXiv preprint arXiv:2509.20846*, 2025.

[43] Helmut Lütkepohl. *New Introduction to Multiple Time Series Analysis*. Springer, Berlin, Heidelberg, 2005. doi: 10.1007/978-3-540-27752-1.

[44] Thomas Verma and Judea Pearl. Equivalence and synthesis of causal models. In *Proceedings of the Sixth Conference on Uncertainty in Artificial Intelligence*, pages 255–270, 1990.

[45] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, second edition, 2000.

[46] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

[47] Sören R. Künzel, Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences*, 116(10):4156–4165, 2019. doi: 10.1073/pnas.1804597116.

[48] Alexander G. Reisach, Christof Seiler, and Sebastian Weichwald. Beware of the simulated DAG! Causal discovery benchmarks may be easy to game. In *Advances in Neural Information Processing Systems*, volume 34, pages 27772–27784, 2021.

A Reference benchmark: DGP parameters and illustrations

Table 4: DGP parameters fixed across the reference benchmark (referenced from Section 4). The "Variant" column lists where each parameter applies; "all" indicates the parameter is shared by every DGP variant. "T+S" abbreviates "Trend+Seasonal"; "N+T+S" abbreviates "Nonlinear+Trend+Seasonal".

Parameter	**Value**	**Variant**
Trajectory length T	1000	all
Within-graph variables d	$\{4, 6, 8\}$	all
Maximum lag $\tau_{\max}$	$\{2, 3, 4\}$	all
Edges per graph	$\approx d$ (sparsity calibrated)	all
Innovation distribution	Gaussian, $\mathcal{N}(0, 1)$, homoscedastic	all
Structural weight magnitude	$\sim \mathcal{U}(0.3, 0.8) \cdot \pm 1$	all
Stability constraint	companion spectral radius $\rho < 0.95$	all
Weight seed	`md5(graph_id)`, fixed per graph	all
Drift coefficient slope β_{drift}	0.5 (linear in t/T)	Drift
Nonlinear transform	$\text{sign}(\cdot)\lvert\cdot\rvert^{0.5}$ (recursive)	Nonlinear, N+T+S, CATE Nonlinear
Trend activation per variable	p=0.5 (Bernoulli)	Trend, T+S, N+T+S
Trend amplitude α_i (when active)	$\sim \mathcal{U}(0.5, 2.0) \cdot \pm 1$	Trend, T+S, N+T+S
Active seasonal components per variable	$\{0, 1, 2\}$ with p=$(0.3, 0.4, 0.3)$	Seasonal, T+S, N+T+S
Seasonal period candidates P_k	$\{12, 24, 50, 168\}$ (feasible if $T \geq 3P_k$)	Seasonal, T+S, N+T+S
Seasonal amplitudes $\beta_{i,k}$	$\sim \mathcal{U}(0.3, 1.0)$	Seasonal, T+S, N+T+S
Seasonal phases $\phi_{i,k}$	random per component	Seasonal, T+S, N+T+S
MCAR missing fraction	0.20 (per entry)	MCAR
MNAR threshold quantile	top 0.20 (per variable)	MNAR
CATE treatment type	binary, replaces treatment-node value at each t	CATE Linear, CATE Nonlinear
CATE observational treatment	Bernoulli(0.5) at each t	CATE Linear, CATE Nonlinear
CATE counterfactual rollouts	$\mathbf{1}_T$ (always-on) and $\mathbf{0}_T$ (always-off)	CATE Linear, CATE Nonlinear
Tuning seed (held out)	41	all
Evaluation seeds	$\{42, 43, \ldots, 51\}$	all

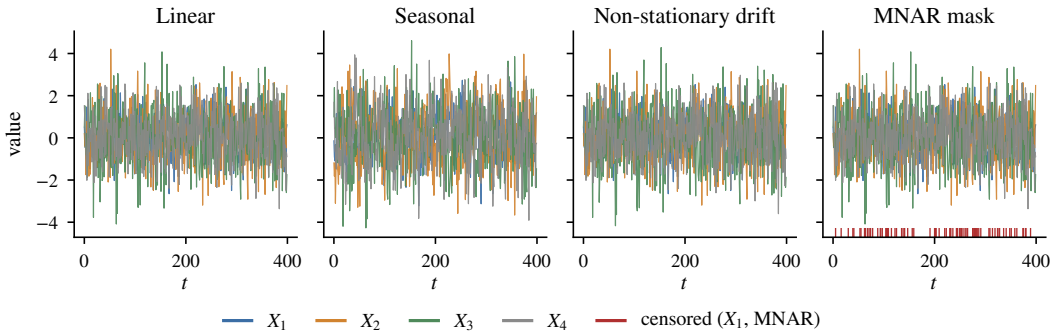


Figure 2: Generated trajectories for graph `d4_tau2` at seed 42 under four DGP variants (T=400 shown for legibility). Graph, weights and noise realization are identical across panels; only the varied axis differs. Red ticks in the rightmost panel mark entries censored by the MNAR mask on X_1.

Table 5: Varsortability audit on the linear reference benchmark; oriented F1 over nine graphs $\times$ ten seeds with hyperparameters, weights and noise realizations fixed across conditions. SortnRegress and RandomRegress are controls, not discovery methods. The DYNOTEARS settings come from an earlier tuning run of the same protocol and were not recomputed after the final reference run.

Condition	v	DYNOTEARS	SortnRegress	RandomRegress
Raw (as shipped)	0.93	**0.99**	0.93	0.79
Standardized	0.50	0.82	0.81	0.79
Δ	-0.43	-0.17	-0.12	0.00

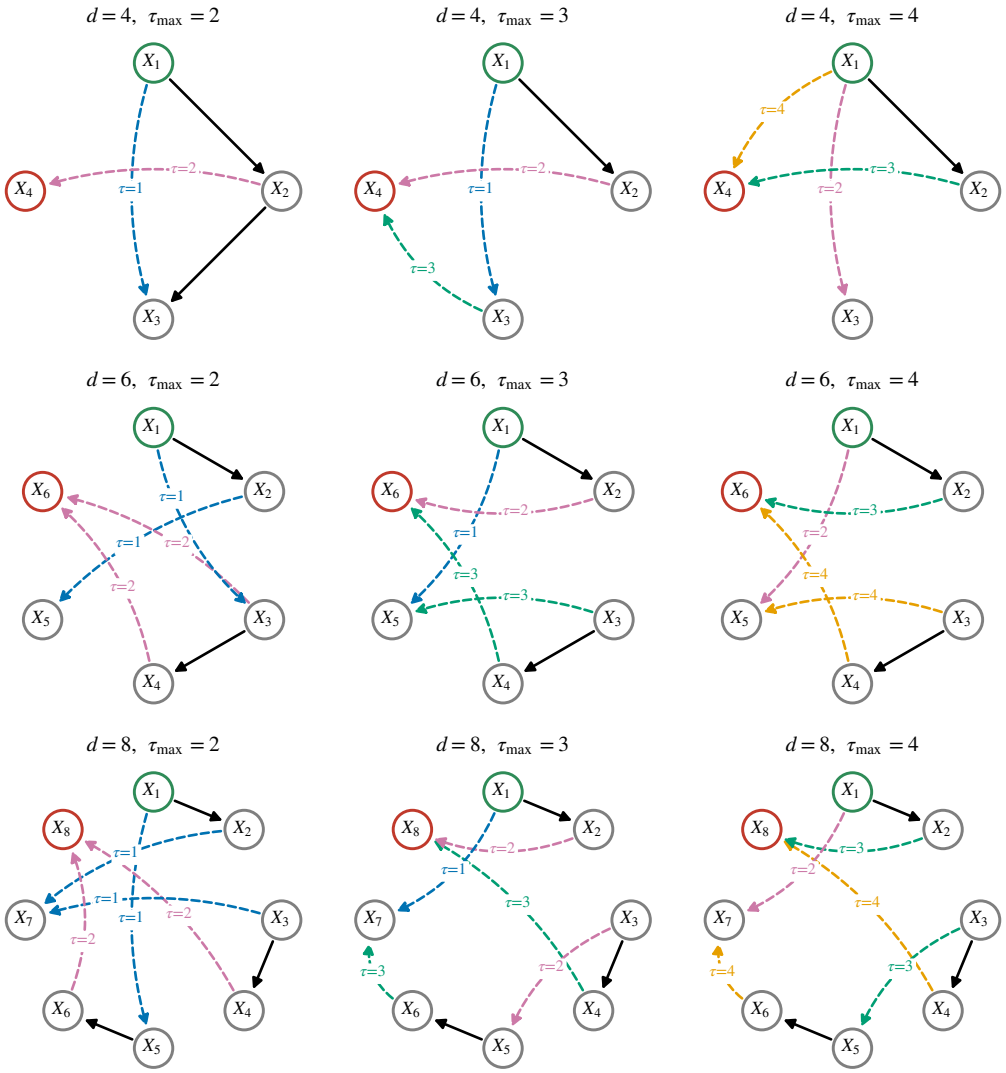


Figure 3: Ground-truth window graphs for the nine reference templates (rows d, columns $\tau_{\max}$). Solid black arrows are contemporaneous edges; dashed arrows are lagged edges, colored and labeled by their lag τ. Green and red outlines mark the treatment and outcome nodes used by the CATE variants. Sparsity is held at $\approx d$ edges per graph.

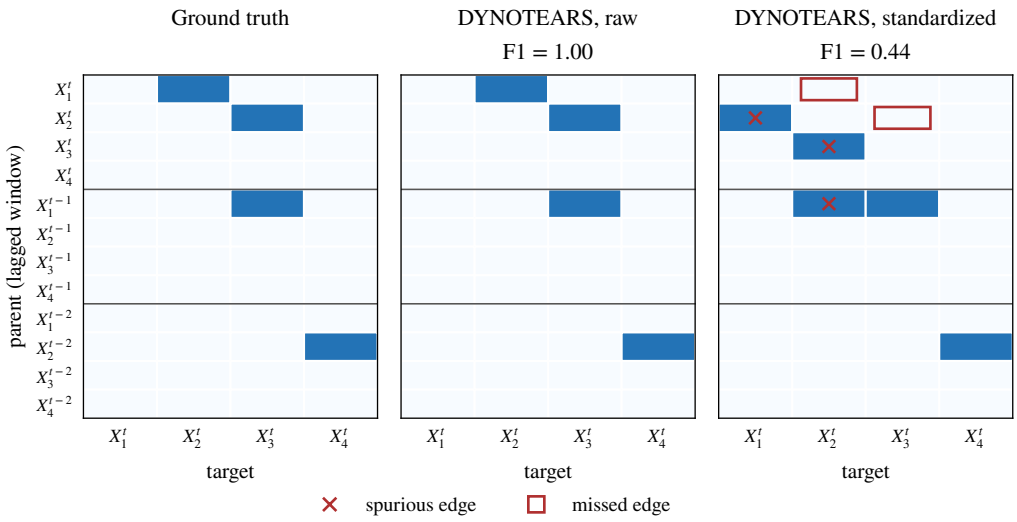


Figure 4: Recovered window graph beside ground truth for `d4_tau2`, linear variant, seed 42, identical hyperparameters. Rows are candidate parents across the lag window, columns are targets at time t; crosses mark spurious edges and open boxes mark missed edges. DYNOTEARS recovers the graph exactly on raw data and misplaces a majority of edges once the marginal-variance ordering is removed.

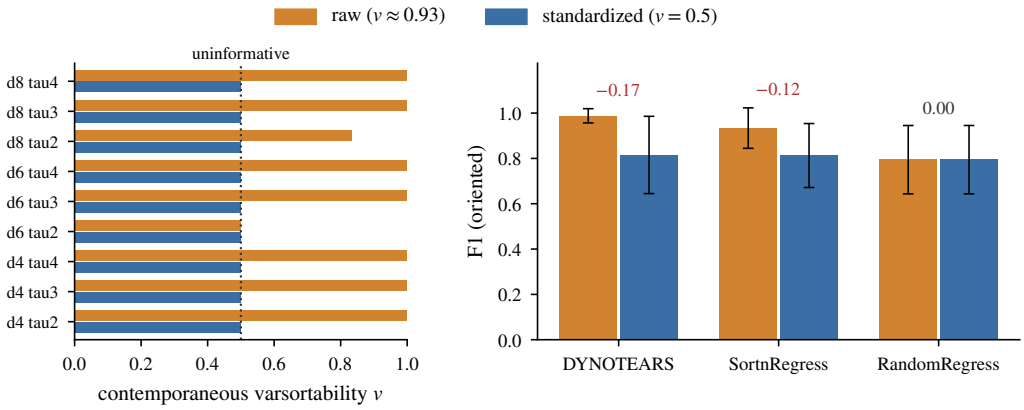


Figure 5: Left: contemporaneous varsortability per graph, raw versus standardized. Right: oriented F1 (nine graphs, ten seeds, fixed hyperparameters and DGP); annotations give the F1 lost to standardization.

B Baseline methods

We provide brief descriptions of each baseline included in the reference benchmark, with implementation source, supported input modalities, and key assumptions. Hyperparameter search spaces appear in Appendix D.

B.1 Causal discovery baselines

VARLiNGAM [9]. Linear non-Gaussian SVAR estimator combining vector autoregression with Independent Component Analysis on the residuals. Identifies the full DAG (both contemporaneous and lagged) under the assumptions of linearity, causal sufficiency, acyclicity, and at most one Gaussian innovation per variable. Implementation: `lingam` package. Does not support missing inputs.

VARLiNGAM_imp. VARLiNGAM with mean imputation applied as a preprocessing step before fitting. Included to enable participation in the MCAR and MNAR variants. The original run's failures for this wrapper are documented, with their cause, in Appendix I.

PCMCI-ParCorr [10]. Constraint-based two-stage algorithm using partial correlation as the conditional independence test. Recovers lagged causal links under causal sufficiency, faithfulness, and stationarity; the ParCorr test additionally assumes linear Gaussian dependencies. Implementation: `tigramite` package. Does not support missing inputs.

PCMCI-GPDC [10]. PCMCI with the GPDC test, which removes the conditioning set by Gaussian process regression and tests the residuals with distance correlation, accommodating nonlinear dependencies. Quadratic-in-T kernel cost restricts applicability to $d{=}4$ in our compute budget. Implementation: `tigramite`. Does not support missing inputs.

DYNOTEARS [19]. Score-based continuous-optimization framework that jointly learns contemporaneous and lagged adjacency matrices via a smooth acyclicity constraint. Assumes linear SEMs; assumes neither Gaussian nor non-Gaussian noise specifically. Implementation: vendored from the `causalnex` package. Does not support missing inputs.

CD-NOD [13]. Constraint-based discovery for heterogeneous and non-stationary data, treating the time index as a surrogate variable to recover contemporaneous causal structure. The algorithm is designed for contemporaneous edge recovery and does not detect lagged edges in its standard form. Implementation: `causal-learn`. Since our reference benchmark graphs contain both contemporaneous and lagged edges, CD-NOD's measured TPR ≈ 0.03 (Table 1) reflects this scope mismatch rather than method failure.

CUTS+ [25]. Joint neural imputation and causal discovery for irregularly sampled or missing-value time series. Uses message passing on a Granger causality graph with delayed supervision. Natively supports missing inputs (`supports_missing=True`); one of the two methods participating in MCAR and MNAR cells in our benchmark.

GrangerF. Linear F-test baseline assessing Granger causality between every pair of variables at each lag, with a configurable significance threshold. Included as the adversarial floor: linear F-tests on marginal lagged dependencies cannot distinguish direct from mediated effects, and the resulting FDR ≈ 0.67 confirms this expected behavior.

B.2 CATE estimation baselines

TARNet [36]. Two-headed neural network with a shared representation trunk and separate treatment-specific heads. It is the variant of the counterfactual regression framework without the integral probability metric balancing penalty.

SNet [37]. Multi-headed architecture with orthogonal information layers designed to disentangle prognostic, propensity-relevant, and treatment-effect-relevant signal. Higher parameter count than TARNet; designed to outperform under high-dimensional confounding.

FlexTENet [38]. Flexible-architecture variant of two-headed CATE estimation with soft information-sharing between control and treatment heads via adaptive layer-sharing. Intermediate complexity between TARNet and SNet.

X-Learner [47]. Meta-learner that fits an outcome model for each treatment arm, regresses the imputed individual treatment effects on covariates within each arm, and combines the two estimates with propensity-score weights. Model-agnostic; we instantiate the base learners as random forests (via EconML).

B.3 Wrapper interface

All causal discovery wrappers expose the unified signature `fit(X, mask=None)` $\rightarrow$ `AdjacencyResult`, with a declared boolean class attribute `supports_missing`. CATE wrappers expose `fit(X, A, mask=None)` $\rightarrow$ `tau_hat`. Methods declaring `supports_missing=False` are skipped on MCAR and MNAR variants and recorded with status `skipped_no_missing_support`, rather than silently returning empty graphs.

B.4 Deferred baselines and reference implementation

Baseline coverage. The benchmark covers eight discovery methods and four CATE estimators across the four discovery paradigms. Recent methods (NTS-NOTEARS [20], stability-oriented temporal DAG learners, CRN [35], G-Net [39], Causal Transformer [40]) require non-trivial wrapper adaptation and are deferred to follow-up; the unified `fit(X, mask)` interface lowers the cost of community-contributed baselines.

Reference implementation. The benchmark was produced by a monolithic `generate_datasets.py` for runtime efficiency; the user-facing OO API in `dtcd.generators` provides the same capabilities. End-to-end equivalence is verified per-cell rather than guaranteed by construction; a unified single-path implementation is on the roadmap.

C Evaluation metrics

We report all metrics over filtered `status=ok` cells unless otherwise noted; skipped and errored cells are tabulated separately in Appendix I.

C.1 Causal discovery metrics

Structural Hamming Distance (SHD). The number of edge additions, deletions, and reversals required to transform the predicted adjacency $\hat{A}$ into the ground-truth adjacency A^*:

$$\mathrm{SHD}(\hat{A}, A^*) = \sum_{i,j,\tau} \mathbb{1}\left[\hat{A}_{i,j,\tau} \neq A^*_{i,j,\tau}\right] - \tfrac{1}{2}\, r(\hat{A}, A^*), \tag{2}$$

where $r(\cdot,\cdot)$ counts reversed edges (each contributes one to the raw symmetric difference but is conventionally weighted as a single edit). Lower is better.

Normalized SHD. $\mathrm{SHD}(\hat{A}, A^*)/E_{\max}$, where $E_{\max} = d^2(\tau_{\max}+1)$ is the maximum possible number of directed edges in the unrolled time-graph representation. Reported alongside raw SHD to enable cross-graph comparison at different d and $\tau_{\max}$.

Skeleton F1 / Orientation F1. Skeleton F1 is the F1 score on the undirected adjacency ($\hat{A}_{i,j,\tau} \vee \hat{A}_{j,i,\tau}$ vs. ground truth), ignoring edge direction. Orientation F1 is the F1 score on directed edges, requiring both presence and direction. Higher is better. Tables reporting "F1" without qualification refer to skeleton F1.

True Positive Rate (TPR) and False Discovery Rate (FDR).

$$\mathrm{TPR} = \frac{|\hat{E} \cap E^*|}{|E^*|}, \qquad \mathrm{FDR} = \frac{|\hat{E} \setminus E^*|}{|\hat{E}|}, \tag{3}$$

where $\hat{E}$ and E^* are the predicted and ground-truth edge sets respectively. TPR is recall; FDR is the complement of precision. Higher TPR and lower FDR are better.

AUPRC. Area under the precision-recall curve, computed by sweeping the edge-confidence threshold across the predicted continuous adjacency matrix prior to thresholding. Captures method behavior across the full operating range, complementing the threshold-dependent F1 / TPR / FDR metrics.

C.2 Treatment-effect estimation metrics

PEHE (Precision in Estimation of Heterogeneous Effects). Root mean squared error between estimated and true individual treatment effects, averaged across timesteps and trajectories:

$$\text{PEHE} = \sqrt{\frac{1}{T \cdot N} \sum_{t,n} \left(\hat{\tau}_{t,n} - \tau^*_{t,n}\right)^2}, \tag{4}$$

where $\tau^*_{t,n} = X^{\text{do}(A=1)}_{t,n} - X^{\text{do}(A=0)}_{t,n}$ is the ground-truth ITE recovered exactly from cached-noise counterfactual rollouts, and $\hat{\tau}_{t,n}$ is the estimator's prediction. Lower is better. PEHE evaluation against ground-truth ITEs is the metric DTCD's counterfactual-twin construction was specifically designed to enable.

ϵ-ATE. Absolute error in the estimated average treatment effect:

$$\epsilon\text{-ATE} = \left| \frac{1}{T \cdot N} \sum_{t,n} \hat{\tau}_{t,n} - \frac{1}{T \cdot N} \sum_{t,n} \tau^*_{t,n} \right|. \tag{5}$$

Reported in supplementary tables; PEHE is preferred in main results because it captures heterogeneity rather than only the average effect.

Runtime. Wall-clock time per `fit` call, in seconds, on the reference hardware described in Appendix L. Reported alongside PEHE to surface accuracy-cost tradeoffs.

D Hyperparameter search spaces

For each baseline we report the Optuna search space actually used by the released code. Each study runs 30 TPE trials per (method, graph, variant) cell on the held-out trajectory (seed 41), except PCMCI-GPDC and DYNOTEARS, which run 5.

Table 6: Hyperparameter search spaces per method.

Method	Hyperparameter	Search Space	Notes
VARLiNGAM	none	—	Nothing tuned
VARLiNGAM_imp	none	—	Mean imputation pre-fit; not tuned
PCMCI-ParCorr	pc_alpha	$\{0.001, 0.01, 0.05\}$	
PCMCI-GPDC	pc_alpha	$\{0.001, 0.01, 0.05\}$	5 trials per cell; $d = 4$ only
DYNOTEARS	lambda_w	log-uniform $[10^{-3}, 1]$	5 trials per cell
	lambda_a	log-uniform $[10^{-3}, 1]$	
	w_threshold	fixed at 0.1	Not tuned
CD-NOD	alpha	$\{0.001, 0.01, 0.05\}$	Contemporaneous-only algorithm
CUTS+	lr	log-uniform $[10^{-4}, 10^{-2}]$	
	lambda_sparsity	log-uniform $[10^{-4}, 10^{-1}]$	
	threshold	uniform $[0.05, 0.5]$	Hard threshold post-training
GrangerF	alpha	$\{0.001, 0.01, 0.05\}$	

E Precision and recall decomposition

Table 7 decomposes skeleton F1 into precision and recall on the Linear DGP. Methods with high precision and lower recall exhibit conservative edge selection; methods with low precision and higher recall exhibit liberal thresholding. The decomposition appears in the main results table (Table 1) under the TPR and FDR columns; this appendix retains a per-method view for readers familiar with the precision/recall framing.

Table 7: Precision and recall decomposition on Linear DGP. Mean over 90 runs per method.

	Precision	Recall	F1
DYNOTEARS	**0.99**	**1.00**	**0.99**
PCMCI_GPDC	0.95	0.67	0.78
VARLiNGAM	0.73	0.80	0.76
VARLiNGAM_imp	0.73	0.80	0.76
PCMCI_ParCorr	0.88	0.65	0.74
CUTS+	0.64	0.87	0.74
GrangerF	0.33	0.65	0.43
CDNOD	0.08	0.03	0.04

F Per-dimension performance breakdowns

Tables 8 and 9 decompose discovery and CATE performance by graph dimension d. DYNOTEARS SHD remains essentially flat across $d \in \{4, 6, 8\}$, confirming that score-based continuous optimization scales gracefully on sparse graphs at this dimension range. PCMCI-ParCorr (0.41, 0.39, 0.56) and GrangerF (1.75, 1.78, 1.97) degrade at $d = 8$, while VARLiNGAM stays near 0.5. PCMCI-GPDC's GP-kernel cost excludes $d \in \{6, 8\}$ from our compute budget. On the CATE side, the Linear DGP gives a mixed picture. X-Learner's PEHE falls from 0.84 at $d = 4$ to 0.61 at $d = 8$, TARNet's goes 0.44, 0.59, 0.44, and SNet's and FlexTENet's rise. Higher dimension therefore does not uniformly degrade estimation, but we have not tested why.

Table 8: Performance breakdown by graph dimension on Linear DGP. Mean ($\pm$ std) over 30 runs per cell (3 τ configurations $\times$ 10 seeds), filtered to `status=ok`. PCMCI-GPDC at $d \in \{6, 8\}$ was excluded by the compute-budget skip.

	SHD			Runtime (s)		
d	4	6	8	4	6	8
CDNOD	1.00	1.03	1.06	0.01	0.01	0.02
CUTS+	0.67	0.36	0.40	0.06	0.06	0.06
DYNOTEARS	0.00	0.02	0.03	0.15	0.41	0.59
GrangerF	1.75	1.78	1.97	0.02	0.04	0.07
PCMCI_GPDC	0.38	-	-	41.78	-	-
PCMCI_ParCorr	0.41	0.39	0.56	0.13	0.27	0.51
VARLiNGAM	0.55	0.47	0.53	0.15	0.27	0.44
VARLiNGAM_imp	0.55	0.47	0.53	0.16	0.29	0.43

Table 9: CATE performance by dimension on Linear DGP. Mean ($\pm$ std) over 30 runs per cell (3 τ configurations $\times$ 10 seeds).

	PEHE		
	d=4	d=6	d=8
TARNet	**0.440 $\pm$ 0.222**	**0.594 $\pm$ 0.127**	**0.438 $\pm$ 0.097**
SNet	0.458 $\pm$ 0.218	0.750 $\pm$ 0.143	0.814 $\pm$ 0.076
FlexTENet	0.478 $\pm$ 0.215	0.753 $\pm$ 0.132	0.756 $\pm$ 0.091
X-Learner	0.841 $\pm$ 0.155	0.820 $\pm$ 0.112	0.611 $\pm$ 0.060

G DYNOTEARS robustness to nonlinear mechanism variants

To probe the robustness boundary of DYNOTEARS' continuous DAG-learning objective under nonlinear DGPs, we ran a targeted experiment varying only the structural mechanism. Table 10 compares three mechanisms on all nine reference graphs across the ten evaluation seeds (n=90 cells per mechanism), holding the graph, weights, noise realization, and DYNOTEARS hyperparameters fixed; the hyperparameters are the values tuned for each graph's Nonlinear cell. The mechanisms are Linear (identity), Nonlinear ($\text{sign}(x)|x|^{0.5}$, strictly monotonic and saturating), and a non-monotonic alternative $\sin(x)$.

As Table 10 shows, the strictly monotonic saturating transform leaves DYNOTEARS' F1 at the Linear baseline (0.99 in both cases). Substituting a non-monotonic but bounded transform produces measurable but bounded degradation (F1 falls to 0.88 and mean SHD rises from 0.11 to 1.14), concentrated on a few graphs (mean SHD 3.0 on d=6, $\tau_{\max}$=4, 2.3 on d=8, $\tau_{\max}$=4, and 2.2 on d=4, $\tau_{\max}$=2, against at most 1.1 on the other six). We additionally evaluated a strictly monotonic but amplifying transform $x \cdot |x|$. Recursive composition of this mechanism inflates trajectories by orders of magnitude (peak $|x|$ above 10^4 on four of the six graphs with $d \geq 6$, up to 9×10^7 at T=1000), which turns these cells into a numerical-conditioning test for any linear-regression-based discovery method rather than a structural-recovery test. DGP specifications that couple recursive amplification with linear-regression-based discovery wrappers should therefore include a stationarity guard; this is on the roadmap for future versions of the library. The experiment is reproduced by `scripts/run_nonlinear_mechanisms.py`.

Table 10: DYNOTEARS performance under different recursive nonlinear mechanisms. Mean ($\pm$ std) over 90 cells per mechanism (nine graphs $\times$ ten seeds). Hyperparameters held fixed at each graph's Nonlinear-cell tuning.

Mechanism	Property	SHD (↓)	F1 (↑)
Linear (x)	identity	0.10 ± 0.30	0.99 ± 0.02
Nonlinear (sign$(x)\|x\|^{0.5}$)	monotonic, saturating	0.11 ± 0.38	0.99 ± 0.03
Non-monotonic ($\sin x$)	non-monotonic, bounded	1.14 ± 1.32	0.88 ± 0.15

H Drift variant results

Table 11 reports performance on the Drift variant of the DGP, in which structural weights vary linearly with t/T. Performance is within 0.08 SHD of Linear for every method, consistent with the view that methods fitting the average structural relationship recover it whether the relationship is strictly time-invariant or smoothly time-varying.

Table 11: Causal discovery performance on Drift DGP. Mean ($\pm$ std) over 90 runs per cell.

	Drift		
	SHD	TPR	FDR
DYNOTEARS	**0.04 ± 0.07**	**0.98 ± 0.05**	**0.01 ± 0.04**
PCMCI_GPDC	0.38 ± 0.17	0.67 ± 0.12	0.06 ± 0.12
VARLiNGAM	0.51 ± 0.36	0.81 ± 0.14	0.27 ± 0.17
VARLiNGAM_imp	0.51 ± 0.36	0.81 ± 0.14	0.26 ± 0.17
PCMCI_ParCorr	0.45 ± 0.16	0.65 ± 0.07	0.12 ± 0.13
CUTS+	0.48 ± 0.22	0.81 ± 0.33	0.33 ± 0.26
GrangerF	1.84 ± 0.57	0.65 ± 0.07	0.67 ± 0.09
CDNOD	1.04 ± 0.11	0.03 ± 0.06	0.92 ± 0.18

I Failed run error distribution

Across the 6,480 (method, graph, variant, seed) combinations attempted, 4,980 succeeded, 960 were skipped because the method declared `supports_missing=False` on a missingness variant, 540 were skipped on PCMCI-GPDC at $d > 4$ by the compute-budget guard, and none raised an exception. Table 12 aggregates these counts by method.

The first run of this benchmark, before the wrapper fix, had 171 errors, all in VARLiNGAM_imp (4,809 completed rows in total). They were not specific to missing data. They occurred on complete data as well (15 of 90 Linear cells), and they traced to the wrapper defect described below rather than to the imputed covariance matrices. Table 13 shows the per-variant counts after the fix. CUTS+, the only other method that supports missing values, completed all 90 cells under both MCAR and MNAR.

Table 12: Aggregate failure mode counts by method, summed across all nine DGP variants.

	Completed	Skipped (no missing support)	Skipped (compute budget)	Errored
CDNOD	630	180	0	0
CUTS+	810	0	0	0
DYNOTEARS	630	180	0	0
GrangerF	630	180	0	0
PCMCI_GPDC	210	60	540	0
PCMCI_ParCorr	630	180	0	0
VARLiNGAM	630	180	0	0
VARLiNGAM_imp	810	0	0	0

Table 13: VARLiNGAM_imp completion vs. error counts by DGP variant, after the wrapper fix. Each variant has 90 attempted runs (9 graphs $\times$ 10 seeds).

status	Completed	Errored	Total attempted
Linear	90	0	90
Nonlinear	90	0	90
Drift	90	0	90
Trend	90	0	90
Seasonal	90	0	90
Trend+Seasonal	90	0	90
Nonlinear+Trend+Seasonal	90	0	90
MCAR	90	0	90
MNAR	90	0	90

Root cause and correction. The VARLiNGAM_imp failures in our first run were traced after submission to a wrapper defect, not to the method: `lingam.VARLiNGAM` selects the VAR lag order by BIC, so its adjacency array can be shorter than $\tau_{\max}+1$, and our wrapper indexed past its end. Because the failing cells were silently dropped from means, and they were exactly the cells where BIC chose a short lag order, the VARLiNGAM_imp scores we first obtained were survivorship-biased upward (nonlinear variant: oriented F_1 0.85 to a corrected 0.76 under the audit protocol). The wrapper is fixed, and the regeneration reported in this paper completes all nine variants with zero failures. On standardized data, and on raw data from the four stationary variants, VARLiNGAM and VARLiNGAM_imp then agree in every cell, as they should, since imputation is a no-op on complete data. On raw data from the five trend, seasonal, and drift variants they differ in 125 of 450 cells, because the VARLiNGAM wrapper standardizes its input before fitting while the imputation wrapper fits on the raw scale.

CUTS+ empty graphs. In 3 of the 81 CUTS+ cells (d4_tau4 Linear, d4_tau4 Drift, and d6_tau3 Seasonal) the tuned setting predicts no edges on any of the ten seeds, which scores $\mathrm{F}_1 = 0$ for the cell and lowers the Linear and Drift means in the main tables. The tuning is deterministic, but which cells collapse depends on the tuning trajectory: the first run collapsed in four other cells.

Determinism of the regeneration. The reference benchmark is seeded end to end. The Optuna TPE sampler is seeded from the tuning-cache key, which covers the method, cell, tuning seed, trial budget, and search-space version, and the CATE estimators receive the evaluation seed. We reran selected cells (four discovery cells with ten seeds each, and two CATE estimators with three seeds) in separate processes and obtained identical rows, with identical tuning caches. We have not tested other machines or package versions; the environment (Python 3.12.13, package versions) is recorded in `environment.json` in the repository.

J Scalability beyond the reference benchmark

The reference benchmark holds $d \in \{4, 6, 8\}$ and T=1000 for tractability. Table 14 extends the sweep to $d \in \{4, 8, 16, 32\}$ and $T \in \{1000, 5000, 10000\}$ at $\tau_{\max}$=3, on Erdős-Rényi window graphs with $\approx d$ edges, three seeds per cell. Only methods whose implementation is vendored here or depends on core requirements are included, so the sweep reproduces without the optional extras.

The graph is drawn once per d and held fixed, so variation across seeds reflects the noise realization only, consistent with the protocol in Section 4.

Table 14: Scalability sweep at $\tau_{\max}$=3, three seeds per cell (one seed at d=32, T=10000). "std." is the variance-standardized condition of Section 4.3. Runtime is single-core wall clock for the raw condition.

		DYNOTEARS F1		SORTNREGRESS F1		DYNOTEARS
d	T	raw	std.	raw	std.	runtime (s)
4	1000	1.00	1.00	0.70	0.85	0.2
4	5000	1.00	1.00	0.70	0.55	0.3
4	10000	1.00	1.00	0.55	0.85	0.4
8	1000	1.00	0.82	0.70	0.75	0.3
8	5000	1.00	0.82	0.74	0.71	0.6
8	10000	1.00	0.82	0.74	0.71	1.3
16	1000	1.00	0.81	0.94	0.83	1.0
16	5000	1.00	0.81	0.98	0.73	1.9
16	10000	1.00	0.81	0.98	0.71	3.3
32	1000	1.00	0.91	0.94	0.76	5.6
32	5000	1.00	0.92	0.92	0.73	7.0
32	10000	1.00	0.94	0.98	0.67	13.4

Two observations. First, DYNOTEARS attains $F_1 = 1.00$ on raw data at *every* cell, including d=32, T=10000. This is not evidence of favorable scaling. SORTNREGRESS also rises with dimension on raw data, which indicates that the marginal-variance shortcut becomes *more* informative as graphs grow, because longer structural paths accumulate more variance separation. Scaling the benchmark up without standardizing would therefore have made it easier, not harder. The standardized condition is the one in which the scaling question is meaningful, and the full grid below is read that way.

Full grid. Table 15 and Table 16 extend the sweep to all nine methods, $d \in \{4, 8, 16, 32\}$, $T \in \{1000, 5000\}$, and $\tau_{\max} \in \{3, 6, 12\}$ in both conditions, three seeds per cell: 1,296 method-cells in total, every one of which completed with status `ok` (no timeouts against a 900 s per-fit budget; slowest single fit 193 s). The sweep runs every method at library defaults, unlike the tuned reference benchmark, and the random-graph family here is sparser and lower-varsortability than the reference graphs (contemporaneous edge fraction 0.25 to 0.31 against 0.38 to 0.50; per-cell v spans 0.0 to 1.0 and is recorded in the released CSVs), so absolute F_1 is not comparable across the two tables and cross-method comparison is not meaningful here. We therefore report the grid as *within-method* scaling plus runtime.

Table 15: Standardized-condition F_1 of the leading method across the full grid, mean over T and three seeds. The minimum over all twelve $(d, \tau_{\max})$ cells is 0.91: once the variance shortcut is removed, accuracy does not degrade with dimension, trajectory length, or lag order.

DYNOTEARS F_1 (std.)	d=4	d=8	d=16	d=32
$\tau_{\max}$=3	1.00	0.94	0.92	0.98
$\tau_{\max}$=6	1.00	0.96	0.95	0.91
$\tau_{\max}$=12	1.00	1.00	0.97	0.98

Two default-configuration rows require a caption rather than a comparison. At library defaults CUTS+ returns an empty graph on this family (the wrapper's edge threshold zeroes every coefficient at the default learning rate), which forces $F_1 = 0$ arithmetically; at the hyperparameters the reference benchmark tuned for it, spot checks on the same cells give 0.82 at d=8 and 0.86 at d=16. Its default-configuration row is an artifact of our wrapper's defaults, not a property of the method or of scaling. CDNOD, by contrast, does predict edges at defaults and its low score reflects a genuine method-DGP mismatch on a sparse linear family that favors SVAR learners.

Varsortability as an instrument. Because v varies naturally across this random-graph family, the sweep doubles as a check on the measure itself. Over the 72 raw/standardized cell pairs, the F_1

Table 16: Mean seconds per fit at the largest dimension (d=32, raw condition; standardized is comparable). The binding runtime constraint shifts with lag order: VARLiNGAM is flat in $\tau_{\max}$ and binds at $\tau_{\max}$=3, PCMCI-ParCorr grows roughly linearly in the window and binds at $\tau_{\max}$=6, and GrangerF grows fastest and binds at $\tau_{\max}$=12.

Seconds per fit, d=32	$\tau_{\max}$=3	$\tau_{\max}$=6	$\tau_{\max}$=12
DYNOTEARS	5	9	16
VARLiNGAM	25	24	21
PCMCI-ParCorr	22	47	118
GrangerF	14	45	187

that SORTNREGRESS loses to standardization is monotone in v: -0.235 where $v = 0.0$ (variance ordering anti-correlates with causal order, so removing it helps), $+0.016$ near $v = 0.5$, and $+0.234$ for $v \in (0.9, 1]$ (Pearson $r = 0.77$). RANDOMREGRESS shows exactly zero change in every cell, and the scale-invariant methods are identical throughout. We note the scope of this check: v measures whether variance ordering matches causal ordering, which is precisely what SORTNREGRESS exploits, so the tight tracking validates the measure and confirms that standardization removes exactly what it targets, rather than independently proving that real methods ride the shortcut; that evidence is the reference-benchmark contrast of Section 4.3, and consistent with being a method rather than a pure exploiter, DYNOTEARS tracks v far more weakly ($r = 0.40$).

K An external equation family: TimeGraph C1

The limitations section notes that a synthetic generator's mechanism class tends to resemble the model forms prevailing when the generator is written. To test the reference results against an independently authored family, we evaluated the same methods on data produced by the C1 generator of TimeGraph [5], a benchmark published before DTCD and designed without reference to it. For transparency we note that TimeGraph is earlier work by the present authors; its equations nonetheless constitute an external family in the relevant sense, since neither the mechanisms nor their parameters were chosen with DTCD's reference benchmark in mind.

Setup. The generator class was imported unmodified from the published TimeGraph repository, and ground truth was parsed from its equation strings and verified against the known structure of each configuration. C1 composes per-node structural terms drawn from sines, cosines, squares, and cubes with a per-channel linear trend, a two-harmonic seasonal component with period 12, and Gaussian noise. We ran all nine published configurations ($d \in \{4, 6, 8\}$, maximum lag in $\{2, 3, 4\}$) at T=1000 with two seeds per cell, in raw and standardized conditions, with per-cell varsortability recorded, under the same oriented window-graph F_1 as the reference benchmark. Three variants separate the exogenous components. The *full* variant uses TimeGraph defaults (trend strength 0.01, seasonal strength 0.5); *seasonal* sets the trend to zero; *mechanism* sets both to zero, a small-amplitude regime in which the sine mechanisms are effectively linear and the even-function edges carry little signal. DYNOTEARS received an oracle-tuned advantage, a 3×2 regularization grid selected on a held-out seed per configuration and condition in the full and seasonal variants, while every other method ran at library defaults. All 108 method-cells completed with zero failed fits.

Family dependence. On the seasonal variant, the fair isolation of the equations, oracle-tuned DYNOTEARS reaches 0.21 raw against 0.99 on the reference linear family, and every untuned method sits at or near the RANDOMREGRESS floor. The reason is mechanistic rather than a matter of tuning. C1's edges are sines, cosines, squares, and cubes, and the even functions among them carry no linear signal, so linear-SVAR and partial-correlation machinery cannot represent most of the graph. Benchmark scores are therefore properties of a (method, family) pair, and the near-ceiling reference results do not transfer to an equation family the generator's authors wrote for a different purpose.

Varsortability as a per-cell diagnostic. The three variants exercise the recorded v metadata across its range. The seasonal variant measures $v = 0.56$, essentially uninformative, and standardization changes little, confirming that this family carries no variance shortcut and that its difficulty lies in the mechanisms. The full variant is perfectly varsortable ($v = 1.00$) because the per-channel trends grow

Table 17: Mean oriented F_1 on the TimeGraph C1 family (nine configurations, two seeds), with contemporaneous varsortability v per variant. †DYNOTEARS is oracle-tuned on a held-out seed in the full and seasonal variants and runs at library defaults in the mechanism variant; all other methods run at library defaults throughout. For reference, tuned DYNOTEARS reaches 0.99 (raw) on the linear reference family of Section 4.3.

	full (v=1.00)		seasonal (v=0.56)		mechanism (v=0.25)	
Method	raw	std.	raw	std.	raw	std.
DYNOTEARS†	0.13	0.19	0.21	0.24	0.00	0.49
SORTNREGRESS	0.14	0.11	0.11	0.09	0.51	0.48
RANDOMREGRESS	0.11	0.11	0.10	0.10	0.44	0.44
VARLiNGAM	0.11	0.11	0.09	0.09	0.44	0.44
GrangerF	0.06	0.06	0.09	0.09	0.30	0.30
PCMCI-ParCorr	0.07	0.07	0.08	0.08	0.31	0.31

with the variable index, yet SORTNREGRESS scores only 0.14; under this degree of non-stationarity the ordering information exists but the regression step cannot exploit it, so high v is necessary but not sufficient for the shortcut to operate.

A second default-configuration artifact. In the small-amplitude mechanism variant, DYNOTEARS at library defaults returns an empty graph on raw data ($F_1 = 0.00$ against 0.49 standardized) because the default penalty dominates coefficients fit to signals with tiny variance. This is the same failure mode documented for CUTS+ in Appendix J, now in a second method, and it reinforces the caption practice adopted there, that default-configuration rows describe configurations, not methods. The scale-invariant methods are identical across conditions in every variant, as they must be.

Scope. Two seeds per cell; absolute F_1 is not comparable across families, since graphs and edge counts differ; the reproduction script ships with the code release.

L Per-method computational cost

The reference benchmark was regenerated on an Apple M4 Mac mini (10 cores, 24 GB) in about 2.6 hours, with four worker processes for discovery and for CATE. Per-fit runtimes are measured under that parallel load, so we compare them across methods and not across machines. Per-method runtime broken down by dimension on the Linear DGP appears in Table 8. PCMCI-GPDC dominates absolute runtime due to GP kernel evaluations, motivating the $d = 4$ restriction in the main benchmark. DYNOTEARS scales modestly with d; PCMCI-ParCorr and VARLiNGAM scale near-linearly. Memory profiling is omitted from the reference benchmark in this version.

M Spatio-temporal extension details

Beyond the within-trajectory framework above, DTCD provides a spatio-temporal module that lifts the structural equations to grids of locations indexed by (t, ℓ, i) for spatial position ℓ and within-location variable i. Locations interact through a configurable spatial kernel $K \in \mathbb{R}^{L \times L}$, with $K[\ell', \ell]$ quantifying the influence of location ℓ' on ℓ. Built-in topologies include 2D and 3D grids and random geometric graphs; built-in kernels include Gaussian, power-law, and step kernels, with a generic adjacency-based kernel for user-supplied spatial structures. The cached-noise rollout mechanism extends naturally to this setting, yielding exact counterfactual twins for spatial interventions (e.g., a hard intervention $do(X_{t,\ell^*,i} = c)$ at a single location ℓ^* propagating to neighboring locations through K). Identifiability guarantees follow analogous classification: the spatial kernel is identifiable under the standard separability assumption that K is constant across time and that within-location and across-location mechanisms factor cleanly. We exclude the spatio-temporal regime from the reference benchmark in Section 4 because no widely-adopted spatio-temporal causal discovery method has emerged in the time-series literature; the module's role is to provide a substrate for follow-up methodological work, not to serve as a current evaluation target.

N Datasheet for DTCD

We follow the structure proposed by Gebru et al. [6].

Motivation

For what purpose was the dataset created? DTCD was created to provide a configurable benchmark with documented identifiability status for evaluating time-series causal discovery and dynamic treatment-effect estimation methods, addressing the empirical gap between synthetic benchmark performance and real-world deployment robustness.

Who created the dataset? Muhammad Hasan Ferdous and Md Osman Gani, Department of Information Systems, University of Maryland, Baltimore County. Funding is disclosed in the Acknowledgments and Disclosure of Funding.

Composition

What do the instances represent? Each instance is a multivariate time-series trajectory $\{X_{t,i}\}_{t=1,i=1}^{T,d}$ paired with its ground-truth contemporaneous and lagged adjacency matrices, the cached noise realization $\mathbf{N}$, the observational mask R when applicable, and (for CATE cells) the counterfactual twin trajectory under a specified intervention.

How many instances are there? The reference benchmark contains $9 \times 9 \times 10 = 810$ discovery instances per method (total 6,480 discovery rows before status filtering) and $9 \times 2 \times 10 = 180$ CATE instances per method (total 720). The framework generates additional instances on demand under any user-supplied configuration.

Spatio-temporal instances. The framework also supports spatio-temporal grid instances of shape (T, L, d) for L locations on a configurable topology, paired with the spatial kernel matrix and cached noise. These are not part of the reference benchmark in this paper but are accessible through the `dtcd.spatial` module.

Are there sensitive attributes or PII? No. All data are synthetic.

Collection process

How was the data acquired? Generated by the DTCD simulator according to the documented DGP (Section 3).

Preprocessing / cleaning / labeling

Ground-truth adjacency matrices are produced jointly with the data. No post-hoc cleaning is performed. Splits are defined at the seed level. Seeds $\{42, \ldots, 51\}$ are used for evaluation, and hyperparameters are tuned on a separate trajectory generated at seed 41, which is never evaluated.

Uses

Has the dataset been used already? For benchmarking the methods in Section 4.

Are there tasks for which the dataset should not be used? Strong performance on DTCD does not guarantee strong real-world deployment performance. DTCD should not be used as the sole evidence for deployment decisions in safety-critical applications.

Distribution and maintenance

License. Library: Apache-2.0. Datasets: CC-BY-4.0.

Maintenance. Issues and feature requests are tracked at `https://github.com/hferdous/Composable_Causality/issues`. Versioned releases follow semantic versioning.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction (Section 1) clearly enumerate DTCD's contributions: a composable framework, exact counterfactual twin generation, identifiability documentation, the reference benchmark across 81 configurations, and open-source release. Section 4 demonstrates these claims empirically, and Section 5 acknowledges where conclusions are scoped.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6.1 explicitly discusses five limitations: synthetic-to-real gap, identifiability boundaries (unestablished cases are named explicitly, and machine-readable tags exist only in the spatio-temporal module), computational scope (including the PCMCI-GPDC $d = 4$ restriction), weight-realization sensitivity (the protocol fixes weights per graph), and the counterfactual sufficiency assumption.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Propositions 1 and 2 (Section 3.2) state identifiability claims for Linear Gaussian VAR and VAR-LiNGAM with their assumptions explicit (causal sufficiency, Gaussianity or non-Gaussianity, acyclicity at each time slice). The recursive nonlinear power-transform mechanism is explicitly marked as identifiability-unknown and presented without an identifiability claim.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 4 fully specifies the benchmark protocol: nine graph templates, nine discovery DGP variants, ten evaluation seeds (42–51), the structural weight seed derivation rule (per graph, fixed across noise seeds), all hyperparameter search spaces (Appendix D), and the structured failure-reporting protocol. Source code, configuration files, and seed lists are released in the public repository, where `bash reproduce.sh` rebuilds every table and figure from the released results and `bash reproduce.sh -full` reruns every experiment.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The library, generation scripts, baseline wrappers, and benchmark data are released under Apache-2.0 (code) and CC-BY-4.0 (datasets) licenses in the public repository linked from the abstract (`https://github.com/hferdous/Composable_Causality`), with documentation and Croissant metadata.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

Justification: Section 4 specifies the optimizer (Optuna TPE), trial budget (30 per cell, reduced to 5 for PCMCI-GPDC and DYNOTEARS), a held-out tuning seed (41) never used for evaluation, tuning objective (F1 against ground truth), evaluation seeds (42–51), and the weight-fixing protocol. Appendix D provides per-method hyperparameter search spaces.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: All tables report mean $\pm$ standard deviation across the ten evaluation seeds (42–51) within each cell. Standard deviations capture noise-realization variance under fixed weights, as documented in Section 4. The structured failure-reporting protocol distinguishes successful runs from skipped or errored runs (Tables 12 and 13).

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The reference benchmark (6,480 discovery cells + 720 CATE cells with hyperparameter tuning) ran on a single Mac Mini (Apple M4, 24GB RAM, no GPU) in approximately 4 hours wall-clock time. Per-method runtime is reported in Tables 3 (CATE) and 8 (discovery).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: DTCD is a synthetic data generation library producing benchmark data with no PII, no human subjects, and no real-world decisions or deployments. The research conforms with the NeurIPS Code of Ethics in all respects.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 6.2 discusses positive impacts (rigorous methodology floor, identifiability documentation reducing miscredited methods) and the absence of foreseeable misuse pathways specific to a synthetic benchmark.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: DTCD releases synthetic data generation code and benchmark numbers, not models with high misuse risk. There is no scraped data, no pretrained generative model, and no individually identifiable content.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

Justification: All baseline implementations are credited via citations in Section 2 and used under their original licenses. The DYNOTEARS solver vendored from CausalNex (Apache-2.0) and the TimeGraph C1 generator (MIT) keep their license notices in the repository. Tigramite (GPL-3.0), causal-learn (MIT), lingam (MIT), CATENets (BSD-3-Clause), and EconML (MIT) are optional dependencies installed from their own distributions and are not redistributed.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: DTCD (the new asset) is documented via this paper, the Datasheet for Datasets in Appendix N, Croissant metadata accompanying the artifact, and per-method docstrings in the released codebase.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: This paper does not involve crowdsourcing or human subjects research. All data are synthetic.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This paper does not involve human subjects research; no IRB approval is required.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs were used for editorial assistance (text refinement, code refactoring suggestions, literature search verification) and as coding assistants that implemented and ran experiment scripts under the authors' direction, as disclosed in the submission form. LLMs were not used as a methodological component of the data generation framework, the discovery or estimation algorithms, or the experimental analysis.